

MANUAL TÉCNICO: DICIONÁRIOS API (v1.0)

Desenvolvedor: Cloves Rodrigues

I. Visão Geral

A DICIONARIOS.dll é uma biblioteca dinâmica baseada em C++/wxWidgets que gerencia quatro bases de dados: Geral, Sinônimos, Dialética e Pronomes. Ela utiliza o padrão **Singleton** para garantir eficiência de memória.

II. Estruturas de Dados (Structs)

O arquivo define quatro "recipientes" (structs) para organizar as diferentes informações linguísticas:

1. **DicionarioGeral**: Armazena o significado padrão.
 - termo: A palavra em si.
 - classe: Classe gramatical (ex: substantivo, verbo).
 - definicao: O texto com o significado.
 2. **Antonimos**: Usado para o dicionário dialético.
 - termo e anttese: A palavra e seu oposto.
 - categoria e contexto: Classificação e exemplo de uso da oposição.
 3. **Sinonimo**: Mapeamento simples de equivalência entre palavra e sinonimo.
 4. **Pronome**: Estrutura detalhada para dados gramaticais.
 - Campos como tipo, caso, pessoa e numero permitem uma análise morfológica completa.
-

III. A Classe Principal: Dicionario

Esta classe é o coração da DLL. Ela é marcada com DICIONARIO_API para que suas funções sejam visíveis para outros programas.

Atributos (Mapas de Dados)

Os dados são armazenados em std::map. Isso permite que, mesmo com milhares de palavras, a busca seja quase instantânea ($O(\log n)$).

- **mapaGeral**: Busca definições por palavra.
- **mapaSinonimos**: Associa uma palavra a um *vetor* (lista) de sinônimos.

- **mapaAntonimos** e **mapaPronomes**: Armazenam as structs correspondentes.

Métodos e Funções Membro

- **static Dicionario* Get()**:
 - **O que faz**: É a porta de entrada do Singleton. Ela garante que só exista uma cópia do dicionário na memória.
 - **Como usar**: Você nunca chama new Dicionario. Você usa Dicionario::Get().
 - **void Inicializar()**:
 - **O que faz**: Carrega todos os dados dos arquivos internos para os mapas na memória RAM.
 - **Como usar**: Deve ser chamada obrigatoriamente uma vez no início do seu programa.
 - **wxString GetDefinicao(wxString termo)**:
 - **O que faz**: Recebe uma palavra, converte para minúsculas e procura no mapaGeral.
 - **Retorno**: Retorna o texto da definição ou uma string vazia se não encontrar.
-

IV. Funções de Carga (Protótipos)

As funções CarregarDicionarioGeral, CarregarSinonimos, etc., são funções auxiliares.

- **Função**: Elas são definidas nos outros arquivos .cpp e servem para "alimentar" a classe Dicionario com os dados brutos (as listas de palavras). Elas são chamadas automaticamente pelo método Inicializar().
-

V. Guia de Uso Prático

Para um programador utilizar esta API em outro projeto, ele deve seguir estes passos:

1. Preparação

Incluir o header e garantir que a DLL esteja acessível.

C++

```
#include "DICIONARIOS.h"
```

2. Inicialização

Antes de qualquer busca, os dados precisam ser carregados na RAM:

C++

```
// Carrega os 4 dicionários  
Dicionario::Get()->Inicializar();
```

3. Realizando uma busca simples

C++

```

wxString busca = wxT("computador");
wxString significado = Dicionario::Get()->GetDefinicao(busca);

if (!significado.IsEmpty()) {
    wxMessageBox(significado, wxT("Resultado"));
}

```

4. Acessando Sinônimos (Lista)

Como uma palavra pode ter vários sinônimos, o acesso é feito pelo vetor:

C++

```

wxString palavra = wxT("feliz");
if (Dicionario::Get()->mapaSinonimos.count(palavra)) {
    std::vector<wxString> lista = Dicionario::Get()->mapaSinonimos[palavra];
    for (const auto& s : lista) {
        // 's' é cada sinônimo individual encontrado
    }
}

```

5. Acessando Gramática (Pronomes)

C++

```

Pronome p = Dicionario::Get()->mapaPronomes[wxT("nós")];
// p.pessoa será "1ª pessoa", p.numero será "plural", etc.

```